

【第2回】 JavaScriptの基礎文法

1. 条件分岐について

1-1. 条件分岐とは

条件分岐とは、「もし〇〇なら、△△する」といったように、指定した条件の結果に応じて処理を実装すること。
JavaScriptに限らず、プログラミングを行う上で、一番基本的な考え方となる。

1-2. if文での条件分岐

if文は条件分岐の最も基本的な記述方法。

if … else if … else の流れで記述することで、複数の条件分岐にも対応することが可能である。

【if文の書き方】

```
if (条件判定1)
{
    条件判定1がtrueの場合の処理
}
else if (条件判定2)
{
    条件判定2がtrueの場合の処理
}
else
{
    それ以外の場合の処理
}
```

※ 条件判定部分では、比較演算子や論理演算子を使用して分岐する条件の判定を行う

1. 条件分岐について

1-3. switch文での条件分岐

switch文は特定の変数に対して、値ごとの処理を分岐して実装することができる。

【switch文の書き方】

```
switch (変数)
{
    case 値1:
        変数が値1の場合の処理
        break;
    case 値2:
        変数が値2の場合の処理
        break;
    case 値3:
        変数が値3の場合の処理
        break;
    default:
        それ以外の場合の処理
        break;
}
```

- ※ switch文では、特定の値に対する条件分岐となる
- ※ switch文では、breakを記述しなければ、全ての処理が実行される

1. 条件分岐について

1-4. 三項演算子での条件分岐

三項演算子を使用すると、特定の条件に対して、**true**・**false**の2種類の処理を**if**文を使用せずに記述することができる。

【三項演算子の書き方】

分岐の条件？ [trueの処理] : [falseの処理]

※ 三項演算子は、一つの条件に対しての処理を記述することができる

※ 三項演算子をネストすることも可能である

【例題1】

変数**age**を定義し、**age**の値が**20**以上の場合には「成人です」、それ以外の場合には「未成年です」というダイアログを表示する**JavaScript**を実装しなさい。

【例題2】

変数**score**を定義し、**score**の値が**90**以上の場合には「**A**判定」、**80**点以上**90**点未満の場合には「**B**判定」、**70**点以上**80**点未満の場合には「**C**判定」、それ以外は「**D**判定」というダイアログを表示する**JavaScript**を実装しなさい。

【例題3】

変数**day**を定義し、**day**の値ごとに曜日をダイアログ表示する**JavaScript**を実装しなさい。(**day = 0**だと日曜日、**1**増えるごとに曜日を加算する)

2. 反復処理 (ループ処理) について

2-1. 反復処理(ループ処理)とは

反復処理とは、同じような処理を何度も繰り返し行うこと。

JavaScriptに限らず、プログラミングを行う上で、条件分岐と合わせて基本的な処理である。

2-2. **for**文での反復処理

for文では、カウンタ変数と終了条件・カウンタの増減値を指定して、終了条件までの回数反復処理を行うことができる。

【**for**文の書き方】

```
for ([カウンタ変数の定義と初期化]; [終了条件]; [増減値])  
{  
  繰り返し行う処理  
}
```

※ **for**文では、カウンタ変数が反復処理の中で自動的にインクリメント(デクリメント)され、終了条件まで処理が繰り返される

2. 反復処理 (ループ処理) について

2-3. `while`文での反復処理

`while`文では、**反復処理の実行条件を指定し、実行条件を満たしている間反復処理を行う**ことができる。

【`while`文の書き方】

```
while ([実行条件])  
{  
    繰り返し行う処理  
}
```

※ `while`文では、実行条件を比較演算子や論理演算子を使用して指定する

2-4. `do・while`文での反復処理

`do・while`文では、**反復処理の実行条件を指定し、実行条件を満たしている間反復処理を行う**ことができるが、`while`文と異なり、**実行条件を後に記述するため、条件の是非に関わらず必ず初回の1回は処理が実行**される。

【`do・while`文の書き方】

```
do  
{  
    繰り返し行う処理  
} while ([実行条件])
```

※ `do・while`文では、スクリプトが上から順に実行されるため、実行条件が`false`でも1回目の処理は必ず実行される

2. 反復処理 (ループ処理) について

2-5. for・of文での反復処理

for・of文では、配列のオブジェクトを使用して、配列の要素ごとに反復処理を行うことができる。

【for・of文の書き方】

```
for (const [配列の要素] of [配列])  
{  
  繰り返し行う処理  
}
```

※ for・of文では、配列の要素を1つずつ使用するため、カウンタ変数や終了条件は不要

2-6. 反復処理内でのbreak・continueについて

反復処理の実装時には、指定した条件で反復処理を中断したり、スキップしたりすることができる。

※ break … 反復処理を中断する
continue … 反復処理をスキップする

2-7. 無限ループに注意する

無限ループとは、反復処理の中で指定している終了条件がプログラム上起こりえないことにより、反復処理が終わらないこと。

反復処理を実装する際には、確実に反復処理が終了条件を迎えるように実装することが重要である。

【無限ループの原因】

- ・ 終了条件の変数・条件比較の結果の値が変わらない
- ・ カウンタ変数をインクリメント(デクリメント)していない

2. 反復処理 (ループ処理) について

【例題4】

1～5を順番にダイアログ表示するJavaScriptを実装しなさい。

【例題5】

1～20までの中の3の倍数の合計を計算し、計算結果をダイアログ表示するJavaScriptを実装しなさい。